Scalable Parallel-in-Time Integration for Equations of Motion

Nathan Chapman[1]
[1]Department of Computer Science, Central Washington University
July 09, 2025

Abstract

Simulating time-dependent physics has traditionally been constrained to using sequential algorithms and thus has not benefitted from advances in parallel computing. Parallel-in-time integration attempts to address this limitation with methods such as the Parareal algorithm, but little work has been done to implement this algorithm using high-performance methods or for Hamiltonian systems. This work provides an implementation of the Parareal algorithm that takes advantage of both the massively-parallel architecture of GPUs as well as the scalability of distributed systems. Benchmarks are additionally provided to showcase the efficiency of this implementation, as well as how cluster topology affects the runtime.

This work is made possible thanks to
my friends for sharing laughs and rants,
Mr. Chris Lacy for making physics phun,
Dr. Brandon Peden for showing me how to be a physicist,
and Dr. Andy Piacsek for never giving up on me.

I wouldn't have been able to do it without you.

Contents

1. Introduction

At the time of writing, a computer can add two numbers in about a nanosecond (or 10^{-9} seconds). Naively extending this, a computer then should take 2 nanoseconds to add a number to the result of an addition of two numbers. Thus three nanoseconds are needed to perform three of these additions, and so on. Considering an actual computer needs to perform many operations that are not directly related to computing the sum of number the user gives it (e.g. running the operating system), these calculations will take (much) more time. Nevertheless, the fact remains that because the result of an addition is needed for the following computation, these operations must be executed sequentially. This constraint means that, for (literally) an astronomical number of additions, the time needed for the full calculation to complete might itself be, astronomical.

The procedure just described is not too disimilar than what occurs in *Euler's method*, which is an incredibly simple technique for approximating the solution to a differential equation. The main idea of this method (as considered in physics) is to produce the value of some physical quantity at some time given its value at a previous time, how that quantity changes over time, and how much time has passed. In fact, this procedure is composed of only two arithermetic executions: an addition, and a multiplication. While this time may be "small" for even billions of executions of the Euler method, what if more are needed?

The work presented here is partitioned into three key parts: Section 2 covers key concepts that are necessary to understand the main purpose of and the methods used to build the following tool, Section 3 covers the details of the implementation of th tool, and Section 4 the analysis of different aspects of the tool.

The two most important concepts presented in Section 2 are: equations of motion, and high-performance computing. Equations of motion (Section 2.1) covers the basics of the mathematical and computional methods that have been used to model physical phenomena, such as the motion of a ball flying through the air. High-Performance Computing (Section 2.2) covers the fundamentals of using mutliple threads to execute multiple calculations simultaneously, including a high-level description of the GPU/Single-Program-Multiple-Thread model of parallelism, as well as the fundamentals of using multiple processes to achieve further parallelization, including a similarly high-level description of how multiple computers (i.e. unshared memory) can be used.

Section 3 comprises the core of this work and presents the Parareal algorithm and how it can be implemented to take advantage of massively-parallel frameworks. Section 3.1 more specifically introduces the Parareal algorithm and presents it in such a way that using high-performance methods is a natural extension. Section 3.2 details how to construct and use a cluster of computers such that the Parareal algorithm can be executed on GPUs across multiple machines that are possibly not even in the same physical location.

Section 4 covers analysis of this implementation. Section 4.1 details the numerical effects of discretization on error/energy drift, stability, and convergence. Section 4.2 considers how using high-performance methods directly affects the runtime and needed memory i.e. time and space complexity. Section 4.3 addresses the core goal of this work, how this implementation affects the runtime needed to approximate motion.

2. Background

As this work lies firmly within in the realm of *computational physics*, the core concepts find themselves spanning physics, math, and computer science. It is the models of physics that give natural processes a mathematical shape so that they may be understood and predicted. Though, it is only for the most spherical of cows in a vacuum that such predictions can be made with pen and paper. These structured representations of nature can be combined with well-defined procedures, i.e. *algorithms*, to be able to make predictions that might actually come true. Whether it be making concrete, testable predictions about how certain types of interacting molecules will tilt when exposed to an electric field at temperatures near absolute-zero [1], or whether or not someone will need their rain jacket in a few days (or 1,000 years). The concepts at the core of the following work are no more than: *equations of motion*, *high-performance computing*.

Physically, **equations of motion** (EOM) (section Section 2.1) are considered in this work to be second order differential equations describing the motion of objects. Another way of interpreting an EOM is as how the acceleration of an object over time depends on the object's position and velocity at that time. The solution to an EOM is simply the position of the object as a function of time, from which the velocity can be derived. The EOMs alone though only provide the behavior of how the position and velocity of the object *changes* over time. In order to uniquely define a path the object takes, initial values for the position and velocity must be stipulated. The EOM together with these initial values, then define an **initial value problem** (IVP). These IVPs have long been studied, but investigations into physics at the most extreme scale have required significantly more resources.

High-performance computing (HPC) (section Section 2.2), in the context of this work, focuses on utilizing two core ideas: **multithreading & GPU computing**, and **multiprocessing & distributed computing**. These ideas contrast *sequential* procedures where the next calculation cannot be started before the previous has finished. Multithreading, and more specifically using graphics processing units (GPUs) to do general purpose computation i.e. GPGPU computing, allow several calculations to be done simultaneously on the same physical hardware i.e. in *parallel*. Futher extending this idea, multiprocessing (not to be confused with multi-*threading*) allows calculations to be executed simultaneously as in the case of several threads, but these calculations "have their own set of knowledge". This seemingly subtle distinction provides the ability for these multiple processes to be executed on *different* physical hardware. These models of parallelism have been used in the past to address the runtime issues arising from simulating complex physical phenomena.

It is through the combination of these core ideas that testable predictions of "extreme-scale" physics can be made. There are many more details and nuances that are not covered here, and many more to improve this work, but that is outside the scope of this discussion. The following presentation of ideas is meant to deliver a functional understanding of the foundational concepts on which this work has been derived.

2.1. Equations of Motion

According to classical mechanics, the motion for any and every object in the universe can be determined for all time using only its current position, current velocity, and the forces acting on it [2]. The foundation on which this principle lies is the *equation of motion* (e.g. Newton's Second Law), which dictates how motion changes in time, or both time and space. These EOMs can be solved via numerical methods, but some methods better suited for specific problems than others, especially when considering the tradeoff between the accuracy of the result and the level of approximation. Additionally, some straight-forward methods solve the EOM by accurately stepping through time, but others first make an estimate of the solution, somehow make corrections to that guess, and then keep doing that until the desired accuracy is achieved.

2.1.1. Initial Value Problems

Take, for exmaple, the motion of a simple pendulum. While the overall motion of bob is determined from length on which it hangs, and the gravity affecting it, the angle at which the bob finds only depends on time. Now, the position of a point on a guitar string does not only change in time, but is also affected by the motion of the points around it. Both of these systems can be modeled by an EOM, but the pendulum can be modeled an **ordinary differential equation** (ODE), and the guitar string can be modeled by a **partial differential equation** (PDE).

The nuances on each of these ideas are better left covered by your friendly neighborhood math department, but the detail that is indeed important to this work is that there are techniques that can transform a PDE to a collection of ODEs. One such procedure is known as the *spectral method*, where by representing the solution to the PDE as a sum of waves (i.e. a Fourier transform), the physics in each dimension only affects the frequency in that dimension [3]. While the soutions to the ODEs would be in so-called "frequency space", applying the inverse Fourier transform on those solutions, achieves the desired solution to the original PDE. Whether it be an ODE, a PDE, or a system of ODEs, when modeling physical phenomena, initial values need to be considered to make any concerete predictions about the future state of a specific object.

For our purposes, an IVP can be thought of as an object with several properties: the acceleration, the initial position and velocity, and the time interval on which you are modeling (which could be unbound e.g. $[0, \infty)$) as shown by the following equation:

$$\left\{ \underbrace{\partial_t^2 \vec{r} = \vec{F}(t, \vec{r}, \vec{v})}_{\text{Acceleration}}, \quad \underbrace{\vec{r}(0) = \vec{r}_0,\ \vec{v}(0) = \vec{v}_0}_{\text{Initial values}}, \quad \underbrace{[0, t_f]}_{\text{time span}} \right\}. \tag{1}$$

In some sense, anything that fits this structure, could be considered an IVP (more on this in Section 3.1). Because initial values "lock in" the trajectory of an object based on the EOM's physics, the IVP can be solved numerically.

2.1.2. Energy Drift & Symplectic Integrators

When it comes to solving EOMs numerically, possibly the biggest factor that should influence the choice of integration method is the length of time on which the EOM is considered. Almost all methods are not inappropriate to use on small scale problems, but some of these methods result in innacurate or even unphysical behavior due to the accumulation of approximation-error. For EOMs, one way to quantify this error is through the idea of **energy drift**.

Outside of considering the energy of the whole universe, the total energy of a (well-defined) system does not change in time. If a system is composed on multiple objects, then the total energy of each object individually can change, but the total energy of all the objects combined is constant. This simple idea provides a reference with which to compare the simulated energy.

The difference between the simulated energy at any point in time and the initial energy acts as a mesasurement of the innacuracy of the simulation at that point in time i.e. the *local error*. A corrolary to this is that the difference between the energy at the end of the simulation and the initial energy serves as a measure of the *global error* as the local error compounds over time. In other words, the energy of the system *drifts* away from the true value as error is compounded.

While almost all methods *can* be used for any problem, some methods result in less energy drift for the same time-step. A specific class of these methods is known as **symplectic integrators**. The underlying reasons for this are out of the scope of this work, but an important note is that even though these methods mititgate the effects of energy drift substantially, they do not yield exactly zero drift [4]. It is these symplectic integration methods that are considered in this work.

2.1.3. Iterative & Multiple-Shooting Methods

When it comes to simulating physics that only depends on spatial behavior, certain methods can be used that aren't immediately available in the time-dependent case. Two classes of these methods are *iterative* and *multiple-shooting* methods. An **iterative method** can be considered a form of "guess and check" algorithm where an initial solution is given, some quantity is calcluated using this solution, and then the process repeats after adjusting the solution to potentially result in a "better" calculated quantity. A **multiple-shooting method** is when one big problem is divided into many problems, each of which only considers a subset of the original domain, and each of these problems is solved independently such that the its values at the domain boundaries agree with those of its neighboors.

An iterative method known as the Hartree-Fock algorithm (also known as the Self-Consistent Field Method) can be used to find the configuration of atoms and molecules that minimizes the system's quantum energy [5]. Likewise, multiple-shooting methods have been used for solving optimal control problems [6]. While these methods have traditionally been used for spatial physics, this work relies on the combination of these principles to solve time-dependent problems.

2.2. High-Performance Computing

When problems get big enough, or more concretely when there is enough data that needs to be processed, traditional computers are unable to complete the task in a reasonable amount of time. When such cases arise, not only are more performant computers needed, but also the methods used to *implement* the calculations need to be changed as well. In essence, **high-performance computing** (HPC) is about performing as many as calculations as possible in the least amount of time. One of the most direct methods to reduce the time needed to perform calculations is to "simply" perform multiple of them at the same time, otherwise known as **parallel processing**. There are several ways in which parallel processing can be acheived; some of which being using a multi-core CPU, a GPU, and multiple physical computers.

Section 2.2.1 covers the basics of utilizing multiple *threads* to perform computations simultaneously. In the case of *multithreading* for HPC, even using multiple threads on a CPU is insufficient as CPUs are only capable of performing, at the time of writing, on the order of 10s of calculations simultaneously. For this reason, the massively-parallel architecture of GPUs has been utilized to performing 10s of *thousands* of calculations simultaneously. Though the extra power does not come freely.

Section 2.2.2 covers the basics of utilizing multiple *processes* and even multiple physical machines to perform *many* calculations simultaneously. Much like multithreading, the potential performance increase from utilizing multiple processes on a single CPU is still limited by its "10s of calculations" ability, and in fact is lower than with multithreading due to processes needing more resources to exist. Unlike multithreading, multiprocessing allows for calculations to be distributed amongst resources that are not even physically located on the same machine, allowing for theoretically *unlimited* performance increases. Though, like multithreading, this utilizing this power does not come without its challenges.

One of the core goals of this work is to combine the power of massive multithreading from GPUs and the unbound potential from distributed computing to solve EOMs. This will allow "extreme-scale", time-dependent problems to be solved in reasonable time.

2.2.1. Multi-threading & GPU Computing

For the purposes of this work, there are three ideas that need to be understood. The first is that data needs to be copied between the CPU and the GPU. The second is how the GPU can read and write that data safely. Finally, the third is how the GPU performs calculations on that data. A note on terminology: when discussing GPGPU computing, the nomenclature refers to the memory and overall system accessible to the CPU as the **host**, and similarly the memory and resources available to the GPU is known as the **device**.

When data is processed and stored by the host in RAM, that data is not automatically accessible to the device. The device has its own memory and can only read and write to it, so any data that is used on the GPU must first be copied to it. When the device has finished its calculations and written the new data to its memory, that data must then be copied from the device to the host. This host-device commuication is orders-of-magnitude slower than the communication between the resources on the device [7], and thus

should be minimized. Note this does not take into account the time needed to *allocate* memory, which only exacerbates the issue.

The data that's transfered between the host and device usually takes the form of an array. Because each thread on the device executes the same program (called the **kernel**), each thread needs to access different elements of the array based on its position relative to the other threads. This follows the *single instruction, multiple thread* (SIMT) model of parallelism. This pattern of indexing is shown in Figure 1.

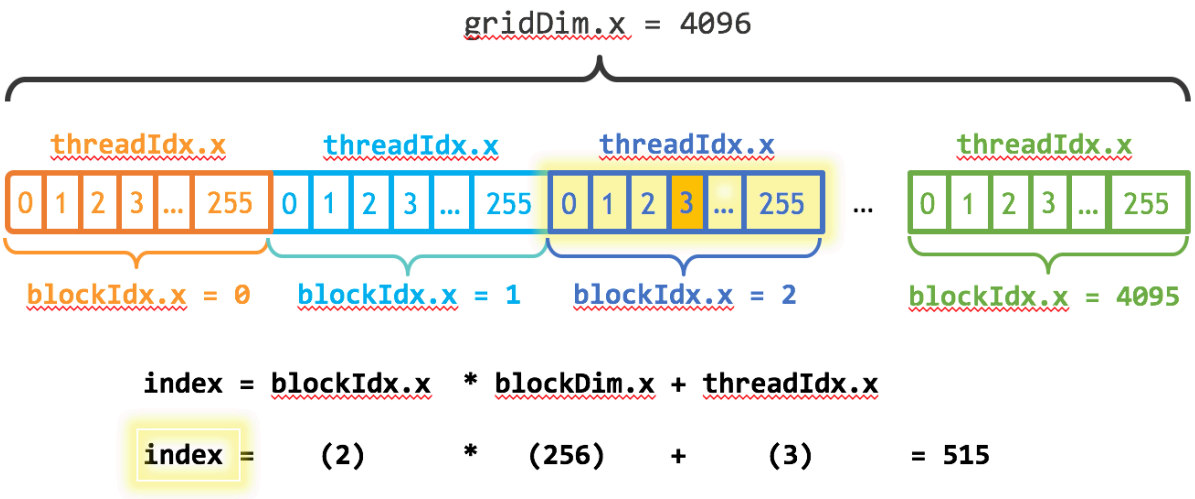

Figure 1: Each thread accesses an index of the array (`index`) based on the thread's location (`threadIdx.x`) in its block, how many threads there are in its block (`blockDim.x`), and the block's location (`blockIdx.x`) in the grid. Image credit [8].

When there are more elements in the array than there are threads on the device, each thread must process multiple array elements. Once each thread is finished writing to its index (either in-place to the original array, or to another array copied from the host), it "jumps over" all the indices that were just written to by all the other threads in all the other blocks, and writes to the next one. The number of indices the thread "jumps", called the *stride*, is determined by the number of threads in each block (`blockDim.x`) and the number of blocks in each grid (`gridDim.x`). This is known as *index striding* and is frequently used in GPU programming to process arrays of arbitrary dimension [9]. This idea is shown in Figure 2.

Once each thread knows its array index, all threads execute the kernel simultaneously. This is where the increased performance comes in. If the program takes T time to execute on a single array element, and there are N array elements, then the total time to calculate sequentially would be TN. Because the device processes each element simultaneously (as long as there are more threads than elements), the total time to calculate is that of a single execuction i.e T. If there are M times as many elements are there are threads, then the total time would simply be MT as each thread processes M elements. Further paralllelization can be achieved by distributing the array elements over multiple devices and machines.

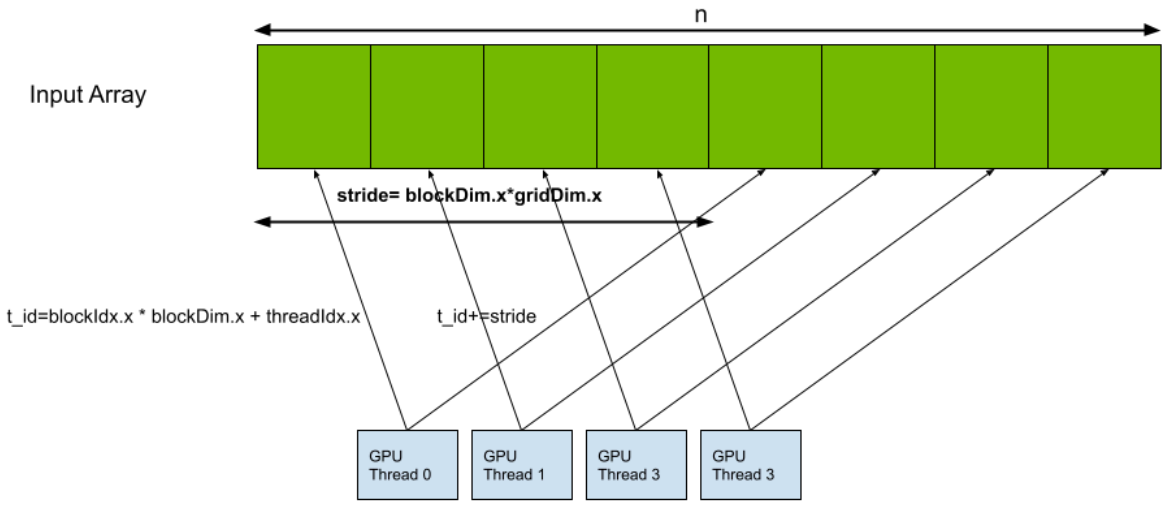

Figure 2: The indices a specific thread processes are based on how many total threads there are. Image credit [10]

2.2.2. Multi-processing & Distributed Computing

The three most important ideas to understand in multiprocessing and distributed computing for this work are: different processes do not have access to the same data, one process can tell another to perform an action, and the time associated with communicating between processes. To distinguish processes and threads, consider two homes A and B each with its own family. Each home represents a process with its associated family members being that process's threads.

Unlike threads, the context, or *memory space*, a process has is distinct from that of other processes. Thus, if something changes in one process, other processes are unaware of the change. Considering the analogy, if home A's phone is moved from the dining room to the living room by Alice who then writes the new location on the fridge, the other family members in home A can all see this change because they all have access to the same fridge. As for home B, not only does the home phone not move, but family B also does not know home A's phone moved, because they do not have access to home B's fridge. It is only when information is explicitly communicated between these processes/homes that the other has this new information.

The communciation of data between between processes is much slower and requires much more overhead than communicating between threads. Considering the analogy, when a family member in home A can't find the phone, they simply go to the fridge for the updated information. If, for some reason, family B needs the location of family A's phone, someone from family A would probably walk over to home B and update them. This takes much more time and resources than going to the fridge in the same home (and even more time and effort is required to update in the next town over!). But what if Bob in home B wants Alice in home A to do something?

Remote Procedure Calls: While there are many methods to communicate information between processes, the most important method for this work involves one processes telling another process to execute some procedure, which is aptly named a *remote procedure call* (RPC). RPC allows one process, such as the one launched by a user running a program, to direct another process to execute some command using its own resources. Though, because each process has its own memory space, any references to objects that don't exist in the *remote* process e.g. variable, libraries, etc., will fail, and references to objects "with the same name" can produce unintended results.

In the analogy, Alice in home A wants to compare the location of her phone to the location of Bob's phone in home B, so she asks Bob "Where is the phone?". Because both homes have phones, but they are in different locations, Alice would answer "the living room" and Bob would answer "the kitchen" because "the phone" is relative to each home. If Alice and Bob wanted to have the same answer, then either they would have to communicate where they want the phone to be and put it there, or refer to the same physical instance of a phone.

Out of the analogy, each remote host effectively needs to be an identical copy of the local host. This can be achieved by each host referring to a shared file system so they all manifestly the same binaries, versions of packages, etc. This needs to happen because an RPC can be thought of as sending a chunk of raw, textual source code to be run on the other process and/or machine. If that source code calls some functionality that is not loaded or otherwise available in that process, that call will error. Thus things like a GPU library must be not only available on each machine, but also loaded on each process.

3. Scaling The Parareal Algorithm

Simulating physical processes has traditionally been done sequentially; even during the modern age of hardware supporting parallel execution, using computers to calculate the evolution of physical phenomena has been sequential. Why haven't scientists just started doing things in parallel? Because of that pesky thing call *causality*; *the ball must go up before it can come down*. Because of this temporal dependence (spatial dependence has had its own workarounds such as the Barnes-Hut algorithm [11,12]), simulation of large-time-scale physics has thus taken a long time to execute. The Parareal algorithm (PA), and other parallel-in-time integration algorithms, have been developed in the last few decades to specifically address this issue [13]. Many details and variations of the Parareal algorithm have been investigated to find and address issues such as stability , convergence rates , application to higher-order differential equations . The main goal of this investigation is to contribute another variation: an implementation of the Parareal algorithm using methods from high-performance computing.

Before the PA can be implemented using these high-performance methods, the algorithm must be decomposed into its central components. The Parareal algorithm begins by partitioning a single IVP into several IVPs on smaller domains via an initial, inaccurate, "root" solution. Then each of the "subproblems" are solved using a sequential, accurate method on different threads at the same time. The final data for each of the subsolutions is then combined with the respective data of the root solution to yield a more accurate (i.e. "corrected") root solution. This new root solution is then used to repeat the process until convergence.

The interpretation of the PA in terms of these recursive subproblems makes the algorithm *almost* embarrassingly parallel; the corrections to the root solution need to be done sequentially. In addition to this structure, the algorithms being evaluated in parallel manifestly depend on simple arithmetic; because of this simplicity, the PA is well-suited to be evaluated on the GPU. Likewise, distributed methods can be combined with GPU evaluation for further parallelization for either a single model (taking advantaged of the recursive nature of the PA) or a system of models.

This chapter begins by casting the Parareal Algorithm in a form that is conducive to being scaled. The main idea is to recast the "magical" mechanism of the PA to something that can be executed recursively. In other words, the PA takes an IVP and produces a collection of IVPs, which can each be given to another instance of the PA. The latter half of of this chapter is devoted to presenting a model for which the scalable version of the PA can be implemented. This model includes how the performance of the PA can increased by using a GPU on a single machine, as well as how to build and use a cluster of machines to further increase performance.

Example: To help understand and clarify the mechanisms of the scalable PA, including the important details that are not explicitly covered in the algorithm itself, an example problem is used. Consider the motion of a thrown ball just after it leaves the hand over the course of 8 seconds (ignoring air resistance). This motion is modeled by: $P = \left\{ \underbrace{\partial_t^2 \vec{r} = \vec{g}}_{\text{Acceleration}}, \quad \underbrace{\vec{r}(0) = \vec{r}_0,\ \vec{v}(0) = \vec{v}_0}_{\text{Initial values}}, \quad \underbrace{[0\text{s}, 8\text{s}]}_{\text{time span}} \right\}$. The machine has 8 cores.

3.1. The Parareal Algorithm

The Parareal Algorithm aimed to solve the problem of physics taking too long to simulate; it didn't.

The main idea of the Parareal algorithm (PA) is to break up a single IVP into many smaller IVPs using some low-accuracy solution, solve those in parallel using high-accuracy methods, correct your initial solution using the sub-solutions, then make a new low-accuracy solution based on the corrected data, and repeat this process until the solution doesn't change. The end result of this procedure is a solution identical to one produced by directly using the high-accuracy method while potentially taking a less time [14]. Because the PA wraps traditional (sequential) solvers, it could be considered a "meta-" or "higher-order" method to solve IVPs.

3.1.1. Preparing the subproblems

Let the second-order initial value problem P be defined such that

$$P = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [t_0, t_0 + \Delta t]\}, \tag{2}$$

and $D = [t_0, t_0 + \Delta t]$ is the closed time interval from t_0 to $t_0 + \Delta t$. The discretization N of P should be determined by the number of available threads N_t such that $N = mN_t$, for some positive integer m. The discretization should be chosen in this manner for maximum performance and efficiency; if $N = mN_t + r$, and $0 < r < N_t$, each thread will solve a subproblem m times until on the $m + 1$ iteration where only r threads would be active while $N_t - r$ threads idle (assuming all threads are synchronized). This type of optimization is sometimes referred to as "*flooding the threadpool*" to mitigate *thread starvation* .

With the discretization decided, partition the time domain D into subdomains D_p such that

$$D_p = \left[t_0 + \frac{p}{N}\Delta t, t_0 + \frac{p+1}{N}\Delta t\right] = \left[t_p, t_{p+1}\right]. \tag{3}$$

Use an fast integration method $\mathcal{G}_0$ (such as the Euler method) to compute initial root solutions $\left\{u_p^0\right\}_p$, $\left\{v_p^0\right\}_p$ defined such that

$$\left\{u_p^0\right\}_p = \{u_0^0, u_1^0, u_2^0, ..., u_{N-1}^0\} \tag{4}$$

$$\left\{v_p^0\right\}_p = \{v_0^0, v_1^0, v_2^0, ..., v_{N-1}^0\} \tag{5}$$

and $\left\{u_p^0, v_p^0\right\} = \mathcal{G}\left(\Delta t, u_{p-1}^0, v_{p-1}^0, \partial_t^2 u\right)$.

Subproblems P_p take the same from as in Eq. 2 (this also means subproblems are themselves, problems), but instead using initial conditions defined by those in the initial root solution. For example, the subproblem P_1 for the second ($p = 1$) subdomain, takes the form

$$P_1 = \left\{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_1^0, \partial_t u(0) = v_1^0, \quad \left[t_p, t_{p+1}\right]\right\}. \tag{6}$$

Algorithm 1 shows the pseudocode of this process for a given IVP and integration algorithm i.e. "propagator", resulting in root solutions and and the collected subproblems. With these subproblems in hand, the PA continues to its next stage: propagating these problems in parallel.

Algorithm 1: PREPARE THE SUBPROBLEMS

INPUT: Second order initial value problem `P`, Coarse solver `C`
OUTPUT: Solution for `P` made from `pos_seq` and `vel_seq`, and array of subproblems `subproblems`
// choose discretization
`N` ← number of subproblems *// e.g. multiple of # of computer cores*
// partition the domain of P into N subdomains e.g. [a, b] -> {[a, c], [c, b]}
`subdomains` ← `partition(P.domain)`
// use coarse solver to get positions and velocities for the root problem
`pos_seq`, `vel_seq` ← `propagate(P, C)`
// create subproblems
for `i` from 1 to `N - 1`
    `subdomain` ← `i`-th domain partition `subdomains[i]`
    `pos0` ← initial position for `i`-th subproblem `pos_seq[i]`
    `vel0` ← initial velocity for `i`-th subproblem `vel_seq[i]`
    `subproblems[i]` ← ivp on `subdomain` with initial values `pos0` and `vel0` for acceleration `P.acc`
return Solution for root problem with `pos_seq` and `vel_seq`, and array of subproblems `subproblems`

Algorithm 1: Prepare the subproblems

Example: Given the example IVP (Eq. 6) and the available threads,

1) There should be 8 subproblems because there are 8 available threads.
2) Create the 8 time sub-domains $[0, 1], [1, 2], ..., [7, 8]$.
3) Use the initial position and velocity to quickly solve the root problem via the Euler method to give a sequence of 8 total positions $\vec{r}_p^0$ and a sequence of 8 total velocities $\vec{v}_p^0$.
4) Use the calculated positions and velocities as initial positions and velocities to create 8 subproblems on the associated subdomains following the form

$$P_p = \left\{\partial_t^2 \vec{r} = \vec{g}, \quad \vec{r}(0) = \vec{r}_p^0, \ \vec{v}(0) = \vec{v}_p^0, \quad \left[t_p, t_{p+1}\right]\right\}. \tag{7}$$

The result of this process is shown in Figure 3.

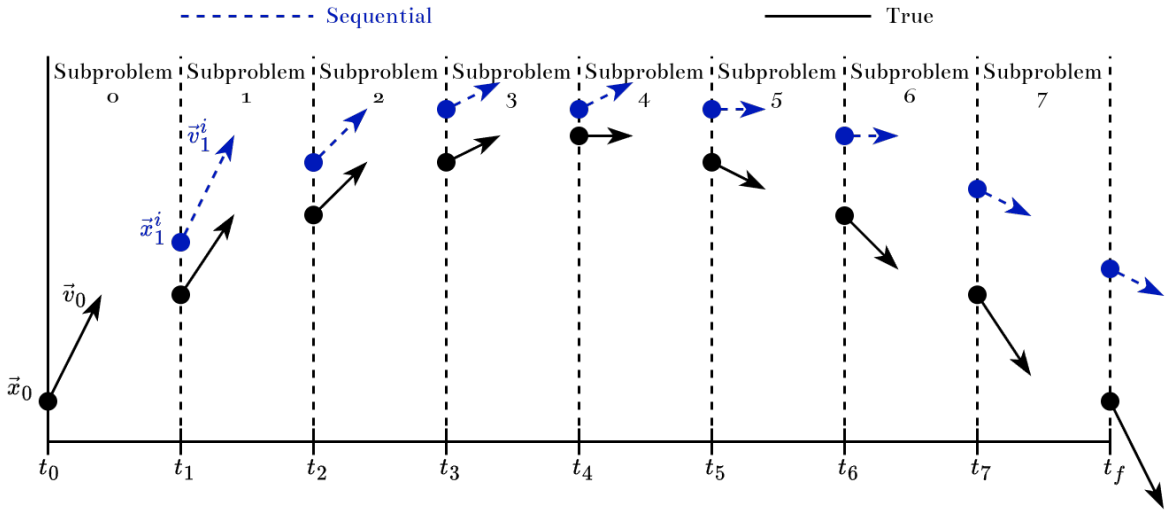

Figure 3: The motion of a ball flying through the air can be partitioned in time to form several initial value problems, each with its own initial position and velocity (upper, blue) determined by a fast integration method. Compared to the true solution (lower, black), this solution is very inaccurate.

3.1.2. Solving the subproblems

Discretizing the domain and propagating initial values as in the previous section constitutes the application of the *coarse propagator* $\mathcal{G}$ to the root problem. Because each of the produced subproblems is independent of the others, each can be accurately solved in parallel using a *fine propagator* $\mathcal{F}$ to reduce the total runtime by a factor equal to the number of subproblems. In other words, if applying $\mathcal{F}$ to a single subproblem has a runtime $\tau_{\mathcal{F}}$, then applying $\mathcal{F}$ to the root problem directly has a runtime $N * \tau_{\mathcal{F}}$ because there are N subproblems, whereas applying $\mathcal{F}$ in parallel only results in a runtime $\tau_{\mathcal{F}}$ because each application of $\mathcal{F}$ executes at the same time.

In general, a **propagator** $\mathcal{P}$ is defined by two key components: its integration algorithm $I_{\mathcal{P}}$, and its discretization $N_{\mathcal{P}}$. More specifically, the propagator $\mathcal{P}$ can be interpreted as a higher-order function mapping integration algorithms and discretizations to functions that, when applied to an IVP, reduce to sequences $\{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t$ of time-position and time-velocity pairs, respectively; the collection of the sequences is called the **solution** S_P of P. The solution can be interpreted as the set of function-graphs (as defined in [15]) of $\vec{r}$ and $\vec{v}$, and is defined such that

$$\mathcal{P}(I_{\mathcal{P}}, N_{\mathcal{P}})(P) = \left\{\{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t\right\} =: S_P. \tag{8}$$

The application of the propagator to the subproblem is the core, or **kernel**, of the PA. While this description of the kernel is useful for understanding, it does not immediately lead to an algorithm that is well-suited for hardware-agnostic implementation (more details in section 3.2.1 on page 19). To that end, the implementation of the kernel presented here is composed of discretizing the subdomain and propagating the initial values separately.

Discretization: To avoid performance losses from each kernel allocating memory, each discretization kernel references a specific, pre-allocated, one-dimensional array δD of length $N_{\mathcal{P}}$. In order for the kernel to generate the appropriate samplings of the domain, δD has its first and last elements pre-populated with the values of the lower and upper bounds of that thread's assigned subproblem such that

$$\delta D = \left\{t_p, [N_{\mathcal{P}} - 2 \text{ arbitrary elements}], t_{p+1}\right\}. \tag{9}$$

With each kernel accessing this data, it can simply calculate and write the domain samples in-place; this process is shown in Algorithm 2.

Algorithm 2: Parallel Discretization Kernel

INPUT: Discretized domain `ddom` of length `N`
OUTPUT: Nothing

```
a, b ← (ddom[0], ddom[N-1])
step ← (b - a) / 2
for i from 1 to N - 2
    ddom[i] ← a + (i - 1) * step
return
```

Algorithm 2: Each discretized subdomain is calculated by uniformly stepping from the lower bound to the upper bound. These results are written in-place.

Propagation: Much like the discretization kernel, the propagation kernel avoids memory allocations by using pre-allocated, pre-populated multidimensional arrays to store the position and velocity sequences; the initial values of the position and velocity for that kernel's subproblem are pre-populated as their respective arrays first element taking the form

$$\{\vec{r}_t\}_t = \left\{\vec{r}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\right\} \quad \{\vec{v}_t\}_t = \left\{\vec{v}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\right\} \tag{10}$$

Otherwise, the propagation kernel is no more than a traditional IVP solver as described in Section 2.1.2, but executed on many different subproblems simultaneously over many threads. This process is shown algorithmically in Algorithm 3.

Algorithm 3: Parallel Propagation Kernel

INPUT: A solver `solve`,
acceleration function `acc`,
sequence of `N` position vectors `pos_seq`,
sequence of `N` velocity vectors `vel_seq`
OUTPUT: Nothing

```
for i from 2 to N - 1
    old_pos, old_vel ← pos_seq[i - 1], vel_seq[i - 1]
    pos_seq[i], vel_seq[i] ← solve(old_pos, old_vel, acc, step)
return
```

Algorithm 3: Each subproblem is solved in parallel using traditional methods.

The Parareal Kernel: In summary, the parareal kernel (Algorithm 4) is launched on each thread simultaneously and uses the fine propagator to populate arrays prepared from the domain and initial values of that thread's problem. The domain is discretized by uniformly stepping from the lower bound of the problem's domain to the upper bound. The initial values are propagated using a traditional integration method (Figure 4). The result of this process is sequences of times, positions, and velocities that solve that thread's problem for different discretizations.

Algorithm 4: The Parareal Kernel

INPUT: a fine propagator `fine`, discretized domain `ddom`,
position sequence `pos_seq`, velocity sequence `vel_seq`
OUTPUT: Nothing

```
Use fine in the discretization kernel to populate to ddom
Use fine in the propagation kernel to populate pos_seq and vel_seq
return
```

Algorithm 4: Put it all together

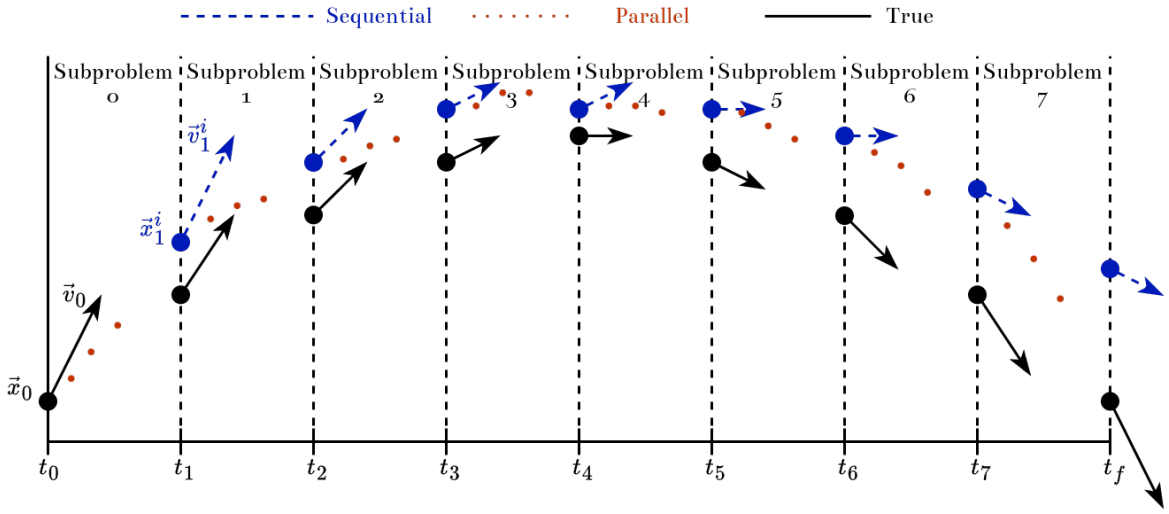

Figure 4: Each thread uses coarse and fine propagators to produce intermediate values (small, red dots) from the initial values (big, blue dots and arrows) of its assigned subproblem. Velocity data does exist, but is neglected here for visual clarity.

Example: For each of the 8 subproblems described by Eq. 7, each taking the form

$$P_p = \left\{\partial_t^2 \vec{r} = \vec{g}, \quad \vec{r}(0) = \vec{r}_p^{\,0}, \ \vec{v}(0) = \vec{v}_p^{\,0}, \quad \left[t_p, t_{p+1}\right]\right\}.$$

1) Define the fine propagator $\mathcal{F}$ as the Velocity-Verlet method with a discretization of $N_{\mathcal{F}} = 2^6 = 64$.
2) Prepare arrays
   2.1) Allocate $3 * 8 = 24$ arrays of length 64 for the fine domains, positions, and velocities.
   2.2) Populate the beginning and end of each domain array with the lower and upper bounds, respectively, of each problem's domain.
   2.3) Populate the beginning of each position array with the initial position of each problem.
   2.4) Populate the beginning of each velocity array with the initial velocity of each problem.
3) Launch the Parareal Kernel with the propagators and prepared arrays
   - **Note:** If there are more problems than threads, the kernel can index stride [9]; more on this in Section 3.2.1.

3.1.3. Solving the root problem

Because the root solution has been calculated via an inaccurate method, it can be made more accurate using the results of the fine propagator. Though the root solution is not corrected only with the results of the fine propagator, but rather by coarsely propagating the root initial values again but adding a corrector determined by a combination of the results of the fine propagator and the previous iteration's root solution. The main idea of this process is known as *Deferred Corrections* [16].

The correction phase, as defined in literature [13], takes the deceptively-simple recursive form

$$u_t^i \coloneqq \underbrace{\mathcal{G}(u_{t-1}^i)}_{\text{predictor}} + \underbrace{\mathcal{F}(u_{t-1}^{i-1}) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{corrector}}, \tag{11}$$

where $u = \vec{r}, \vec{v}$ represents either position or velocity of the root problem. If the coarse propagation terms are collected as $\Delta_i \mathcal{G}_t^i \coloneqq \mathcal{G}_t^i - \mathcal{G}_t^{i-1}$, Eq. 11 can be interpreted as *shooting method in time* [14]. It should also be noted that because the values returned by the coarse propagator are identical in successive iterations i.e. $i \to i+1 \implies \mathcal{G}(u_{t-1}^i) = \mathcal{G}(u_{t-1}^{i-1})$, they can be memoized and not calculated again (see Algorithm 5), leading to performance increases at the cost of storage space [17].

One way to interpret the correction equation is "at face value" as

> *On the current iteration, use the coarse propagator to recommend[1] what this value should be. Then correct it by adding the difference between the fine and coarse predictions from the previous iteration. Record this sum as the actual value.*

An alternative interpretation arises from, effectively, "moving the correction to the top of the loop" as

> *Use the coarse propagator to traditionally evolve the root problem, but with an offset. This offset is initially zero.*

This interpretation changes "*use the propagator, then correct the value*" to "*correct the propagator, then use the value*".

Thus the overall behavior of Eq. 11 could be defined through the explicit recurrence relation

$$\begin{aligned} u_t^i &= \mathcal{G}_t^i(u_{t-1}^i) \\ u_0^i &= u(0). \end{aligned} \tag{12}$$

Here the coarse propagator is effectively parameterized and can vary throughout iterations and times such that $\mathcal{G}_t^i(u_{t-1}^i) \coloneqq \mathcal{G}(u_{t-1}^i) + \Delta_t^i$ and

$$\begin{aligned} \Delta_t^i &= \mathcal{F}(u_t^i) - \mathcal{G}(u_t^i) \\ \Delta_t^0 &= 0. \end{aligned} \tag{13}$$

Eq. 12 has the same structure of a traditional propagation making it much simpler to understand its behavior. This understanding is made even easier when considering Eq. 13 as it allows the Eq. 12 to also define the original coarse propagation of the root problem. In other words, the corrected coarse propagator has a correction of zero on the original iteration. While this interpretation is easier to understand, the result is mechanically identical to Eq. 11.

[1] This terminology is inspired by Giordano & Nakanishi's interpretation of the Gauss-Seidel and Simultaneous Over Relaxation methods in their seminal text on computational physics [18].

Algorithm 5: New Solution Generator

INPUT: Coarse solver `coarse`,

Coarse position array `cpos`, Coarse velocity array `cvel`,

Fine position array `fpos`, Fine velocity array `fvel`,

Root position array `psol[i-1:i, :]`, Root velocity array `vsol[i-1:i, :]`

OUTPUT: Nothing

```
// evaluated at iteration i
psol[i, 0], vsol[i, 0] ← p0, v0
for t from 1 to N-1
    ppred[i, t], vpredl[i, t] ← coarse(psol[i, t-1], vsol[i, t-1])
    psol[i, t] ← ppred[i, t] + fpos[i-1, t] - cpos[i-1, t]
    vsol[i, t] ← vpred[i, t] + fvel[i-1, t] - cvel[i-1, t]
```

Algorithm 5: New position and velocity solutions are generated by coarse propagating the previous solution in the current iteration and combining it with the difference between the fine and coarse solutions from the previous iteration. The new solutions are pushed to a the end of the solution arrays.

Example:

1) Have arrays of positions and velocities at each time for the previous and current iteration; the first element of the current iteration's array is the initial value of the problem, while the rest of empty. Also have arrays of positions and velocities generated from the coarse and fine propagators at each time for the previous iteration.
2) Use the new solution generator algorithm with these arrays and the coarse propagator.
3) The arrays for the position and velocity of the root solution at the current iteration are now populated.

3.1.4. Converging the root solution

Finally, as own in Algorithm 6, launch the parareal kernel (Algorithm 4) to gather the fine solutions for each point in time, and construct the new root solution (Algorithm 5) until the root solution stops changing between iterations. While there are many choices that can serve as valid convergence criteria [14], one of the simplest is:

$$\max_{1 \leq t \leq N-1} |u_t^i - u_t^{i-1}| < \varepsilon, \tag{14}$$

for some threshold ε. Eq. 14 determines convergence when every point in the solution changes by less than some amount between iterations.

Algorithm 6: The Parareal Algorithm

INPUT: Root problem `P`, Coarse propagator `G`, Fine propagator `F`, Convergence threshold `ep`
OUTPUT: Discretized domain `ddom`, Root position sequence `pos`, Root velocity sequence `vel`

```
1 ddom, pos[0, :], vel[0, :] ← Prepare the subproblems via a coarse solution
2 i ← 1
3 while max(changes) ≥ ep
4     fpos[t], fvel[t] ← Launch the parareal kernel in parallel with F, pos[i-1, :], vel[i-1, :] to get
      the fine solutions for subproblem t
5     pos[i, :], vel[i, :] ← Construct new root solutions with G, pos[i-1, :], vel[i-1, :], and
      fpos, fvel
6     changes ← The difference between the current and previous solutions
7 return ddom, pos, vel
```

Algorithm 6: The Parareal Algorithm is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

The magic of the Parareal algorithm lies in its divide-and-conquer approach to solving initial value problems. The "root" problem is sequentially and inaccurately solved to divide it into smaller problems whose initial values are defined by the solution. Those problems are simultaneously and accurately solved in parallel. The root problem is then solved in the same way as before, but at each step, the data is modified by combining the previous accurate and inaccurate solutions. Finally, the new root solution defines new problems, and the loop continues until the the solution has converged.

3.2. Implementing the Parareal Algorithm at Scale

Surely more threads is the answer!

Section 3.1 highlights the PA's nature of being quasi-embarrassingly-parallel i.e. the performance of the algorithm scales with the number of threads, while still being bottlenecked by a periodic sequential process. That being said, the previous discussion ignores the details and nuances of implementing the PA including the actual form of the threads. The primary goal of this work is to provide two new implementation models that both take advantage of increased parallelism and allow easy scaling: using the massively parallel architecture of graphics processing units (GPUs), and the scalability of distributed systems. Instances of these implementations are also provided [19].

The PA as defined in Algorithm 6 does not change in it *what* it does when implemented to use GPUs, but rather *how* it does. There are several issues that arise when utilizing general-purpose GPU computing (GPGPU) such as the GPU needing to wait until the CPU tells it to do something, better performance with less precision, and the restriction to using primitive types like "ints" and "floats". Though, the biggest issue is the need to consider the movement of data between RAM and VRAM, or more generally host memory and device memory; considering unshared memory spaces will be even more important in section Section 3.2.2.

The distributed-based implementation focuses on distributing problems across multiple remote machines. These machines solve their problems simultaneously with the other machines, thus achieving a form of parallelism only limited by the number of accessible machines. These problems could either arise from the coarse propagation of a single root problem, yielding a "problem tree" () where each machine would create-distribute-collect its own set of problems, or if there are multiple "true root" problems e.g. a system of ODEs.

The GPU- and distribution-based methods can be combined to further parallelize solving an initial value problem. If each of the machines available to the distributed network has at least one GPU (a single machine can have multiple; more details in Section 3.2.2), this implementation will automatically identify, manage, and use all of them. Thus these methods can be composed to provide a scalable model of parallel-in-time integration for equations of motion.

3.2.1. Massively Parallelizing the Parareal Algorithm

The GPU-based implementation focuses on three ideas. The first is utilizing the massively-parallel architecture of a GPU to *simultaneously* use orders-of-magnitude more threads than what would be possible with a CPU. The second is considering the movement of data between the host memory (RAM) and the device memory (VRAM). And the last is needing to use primitive data-types. Otherwise, the underlying algorithm is no different than what is presented in Section 3.1.

The PA (Algorithm 6) is only limited by the number of threads at its disposal. When the number of threads is greater than the number of cores, the processor needs to switch thread contexts in order to balance the

evolution of each thread. On a CPU, this context switching is very costly and can lead to drastic decreases in performance [20,21]. On a GPU however, switching thread-contexts is nearly free [22], allowing there to be *many* more threads than processors without sacrificing efficiency. So, it is very beneficial to execute the PA on hardware that not only can efficiently handle many threads, but also have them running at the same time.

All relevant data is first allocated and pre-populated by the CPU on the host. Then the CPU copies that data to the device. Then the CPU tells the GPU to execute the parareal kernel on its copy of the data, producing solution data. The CPU then copies the solution data from the device to the host and recreates the problems. The transfer of data (and the CPU launching the kernel on the GPU) between the host and the device serves as the main performance bottleneck in this process. [22] Dynamic parallelism can be used to launch kernels directly from the GPU, thus circumventing the performance drawbacks of host-device communication [22].

GPUs can only process primitive types of data such as integers, floats, booleans, and other "bits-types". This precludes collecting the problem and solution data in more intuitive forms like one would do when representing them mathematically. In other words, whereas a CPU is happy to handle several boxes each with its own set of elements e.g. domain, acceleration, initial position, and initial velocity, GPUs need this same underlying data to be collected such that all domains are in one box, all acceleration functions are in another box, all initial positions are another, and initial velocities in another. These "boxes" take the form of arrays. It is for this reason, the PA as described in Section 3.1 uses its data as arrays.

So, why is the PA well-suited to be implemented to use GPUs? Because the data is only composed of numbers, it can be simply represented in a GPU-friendly array structure. The massive number of cores on a GPU can simultaneously process these arrays with a much lower cost of switching between threads and problems. And finally, the solution data can be easily copied back to the host. This process is shown in Algorithm 7 and Figure 5.

Algorithm 7: The GPU-Based Parareal Algorithm

INPUT: Root problem `P`, Coarse propagator `G`, Fine propagator `F`, Convergence threshold `ep`
OUTPUT: Discretized domain `ddom`, Root position sequence `pos`, Root velocity sequence `vel`

```
1  i ← 0
2  ddom, pos[i, :], vel[i, :] ← On the host, prepare the subproblems via a coarse solution
3  while max(changes) ≥ ep
4      i += 1
       // fine propagate on the device
5      posd[i-1, :], veld[i-1, :] ← Copy F, pos[i-1, :], vel[i-1, :] to the device
6      fposd[t], fveld[t] ← Launch the parareal kernel to get the fine solutions for subproblem t
7      fpos, fvel ← Copy the fine solutions fposd, fveld to the host
       // coarse propagate on the host
8      pos[i, :], vel[i, :] ← Construct new root solutions with G, pos[i-1, :], vel[i-1, :], fpos, fvel
9      changes ← The difference between the current and previous solutions
10 return ddom, pos, vel
```

Algorithm 7: The Parareal Algorithm is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

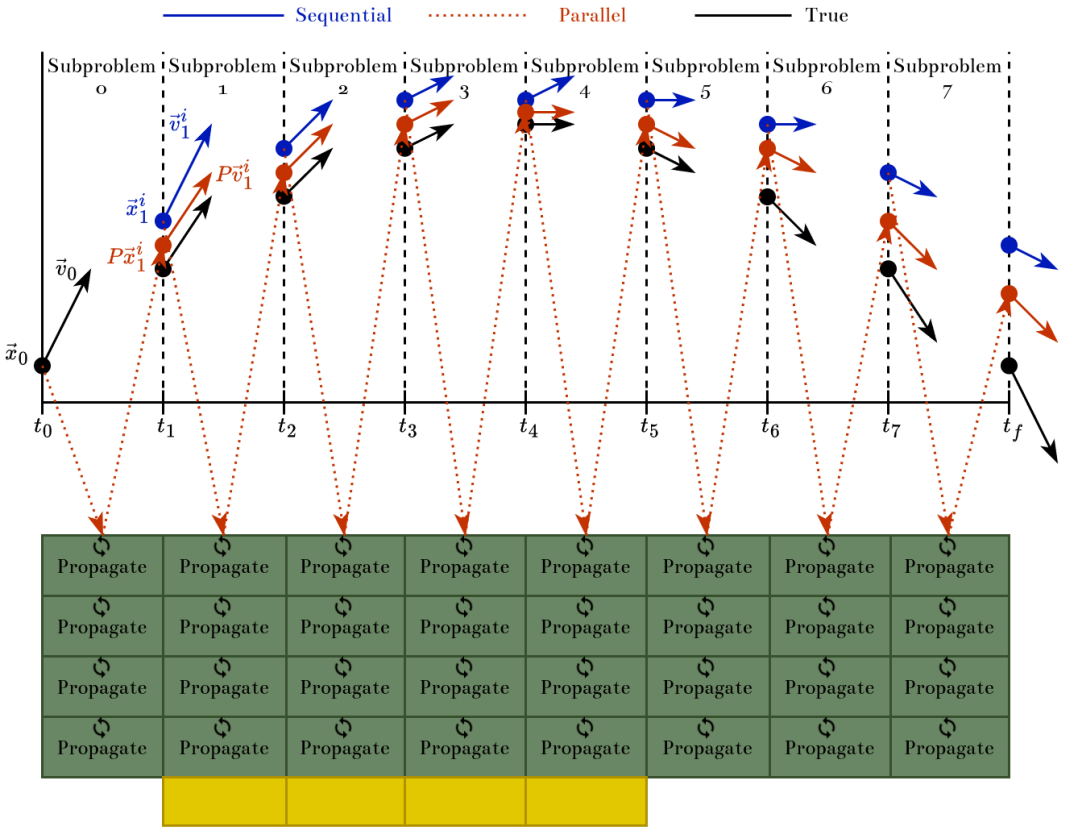

Figure 5: Sequential solutions (top in blue) are sent to the GPU to be finely-propagated (mid in red) in parallel; true solutions (bottom in black) are shown for comparison.

3.2.2. Distributing the Parareal Algorithm

While the PA can be further parallelized using GPUs, the fact still stands that the PA is quasi-embarrassingly-parallel. In other words, each subproblem is independent of the others while each is being solved, and each of these subproblems can be assigned its own thread. So, if there are more threads available, higher performance or accuracy can be achieved. The implementation presented here provides more threads by sending problems to remote machines where they can be run simultaneously; in other words, multiple machines with their own CPUs and GPUs are networked together to form a *cluster* where the work is distributed amongst all machines.

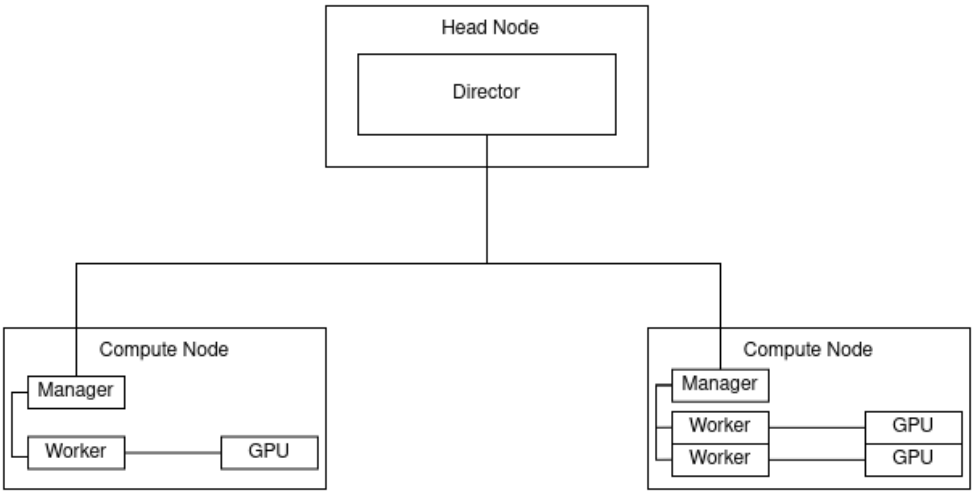

Figure 6: The assumed topology of the cluster presented in this work.

While clusters can take many forms , this implementation considers building a cluster from the ground up in a modular and ad-hoc manner. This way the cluster can theoretically scale without limit. The general strucutre, or *topology*, of the cluster is rather simple: A *head node* runs the *director* process, which tells *worker* processes on other *compute nodes* what to do; this structure is shown in Figure 6. In other words, the director only decides and does not do, while the workers do not decide and only do. While the workers can solve their problems and communicate with every other worker (and the director), only the director can spawn new processes. Once the director has finished spawning and configuring the workers as in Algorithm 8, the director moves on to begin the PA.

Algorithm 8: Prepare the Cluster

INPUT: A collection of hosts not ready to compute
OUTPUT: A collection of hosts ready to compute
1 Director process spawns manager processes on each host
2 Each manager sends the number of devices on its host back to the director
3 Director process spawns worker processes on each host, 1 for each device on that host
4 Pair each worker with a device on its host

Algorithm 8: The cluster can be created and prepared in 4 simple steps.

In order for this implementation to be flexible, the director does not assume any a-priori configuration of any worker nodes. This way any node can be seamlessly introduced to the cluster. The drawback of this flexibility is that the cluster must be created each time. Part of this creation is identifying the available devices in the cluster. To do this, the director first uses a user-given collection of hostnames to spawn a single "manager" process on each of the given hosts.

Algorithm 9: Spawn Manager Processes

INPUT: A list of hosts
OUTPUT: A list of manager process IDs
1 The director spawns manager process on each host
2 The director tells each manager to load the Parareal library

Algorithm 9: Manager processes are created to identify the number of devices on a host, and can manage the network communication between hosts.

Once the managers have been spawned, the director asks them how many devices are on their host. The manager measures this and sends back the information to the director. The director, because it is the only process that can spawn workers, spawns a number of workers on each host equal to the number of devices on it. This process is shown in Algorithm 10.

Algorithm 10: Spawn Worker Processes

INPUT: A list of manager IDs
OUTPUT: A list of worker IDs
1 The director tells each manager to load the GPU library // *provides ability to count devices*
2 The director requests the number of devices on the host from each manager
3 For each host
4 | For each device on the host
5 | | The director spawns a process on the host
6 The director tells each manager to load the Parareal library

Algorithm 10: Worker processes are spawned by the director on each host based on the number of devices available to that host.

Once the workers are spawned on their respective hosts, each of them needs a device. While the fundamental idea of assigning a device to a worker is trivial (as shown in Algorithm 11), the implementation suffers from the fact the a device should not be assigned to a process on a different host! In this implementation, process IDs correlate to the order in which they were spawned e.g. process 2 was spawned second, process 3 was spawned third; in other words, the process IDs are relative to the whole cluster. The device IDs, however, are relative to their host machine. So, care must be taken in order to pair processes and devices on the same host.

Algorithm 11: Assign Devices - High Level

INPUT:
OUTPUT:

```
1 The director tells each new worker to load the GPU library // provides ability to assign devices
2 For each host
3 |  For each worker on that host
4 |  |  The worker assigns an on-host device to itself
```

Algorithm 11: Pairing workers and devices is trivial at a high level

For example

- The director has process ID (pid) 1 and is on its own host.
- Host X has pid 2, and host Y has pids 3, and 4.
- Host X has 1 device with ID 0, and host Y has 2 devices with ID 0 and 1.
- The desired result is
  - Device 0 on host X is assigned to pid 2
  - Device 0 on host Y is assigned to pid 3
  - Device 1 on host Y is assigned to pid 4

One way to address this issue is to create "host objects" by collecting the hostnames, pids, and number of devices for each host. The collection of these host objects, together with their relative connections, would constitute a "cluster object" or "cluster graph". While the actual ids of the devices on each host are what is important, the pattern is the same for all hosts e.g. `devids = 0, 1, ..., ndevs-1`. This way only a single integer needs to be stored instead of a list.

Algorithm 12: Create Host Objects

INPUT: List of hostnames, list of managers, list of device counts
OUTPUT: List of host objects `hosts`

```
For each host
    name ← name of host
    workers ← list of woker IDs on host
    ndevs ← number of devices on host
    hosts[i] ← Host(name, workers, ndevs)
return hosts
```

Algorithm 12: To aid in the pairing of devices and workers, host objects can be created to make sure devices are assigned to workers on the same host.

A host object packages together the worker IDs and number of devices on the same host. Thus devices can be assigned to workers on the same machine simply by iterating through the host objects. This process is shown in Algorithm 13.

Algorithm 13: Assign Devices

INPUT: List of host objects `hosts`
OUTPUT: Nothing

```
Load the GPU library on each worker // provides ability to assign devices
for host in hosts
    workers ← host.workers
    devids ← (0, 1, ..., host.ndevs-1)
    for each pair (worker, devid)
        Assign devid to worker
```

Algorithm 13: A more detailed algorithm of assigning devices to workers on the same host.

Once the devices are assigned, the cluster has been prepared. The director then becomes the driver of the PA, as shown in Algorithm 14. The director begins the PA as it normally would, but instead of either dispatching the parallel propagation to other CPU threads or the GPU, the director gives each worker a problem to propagate in parallel, either on its CPU or GPU. Since this dispatching takes little time, the director then waits until it gets the solutions from the the workers. Finally, the director coarse propagates the root problem with the solutions, checks if the solution has converged, and loops if it hasn't.

Algorithm 14: The Parareal Algorithm at Scale

INPUT: A root problem, coarse and fine propagators, and a convergence threshold
OUTPUT: A solution to the root problem
// on a prepared cluster
The director coarse propagates root problem to get initial solution
While the root solution has not converged
    The director creates problems
    For each problem
        The director sends the problem to a worker
        The director tells the worker to solve the problem using the Parareal algorithm on its GPU
        The director requests the solution from the worker
    The director waits to get all solutions
    The director coarse propagates root problem with corrections to get new solution

Algorithm 14: The Parareal Algorithm can distribute its subproblems amongs multiple machines in order to achieve higher performance.

4. Analysis

While there are many significant aspects of this work that could be analyzed, there are only three that will be considered here. The effects of transferring data betwene the host and the device repeatedly arises only in the implementation and the theoretical nature of this bottleneck is investigated. Similary, the significance of transferring data between hosts is analyzed. And finally, empirical benchmarks are given to highlight the efficacy of the implementation.

Numerical analysis focsuses on measuring the effects of numerical approximation. Some of the most notable effects to conisder are error, stability, and convergence. Instead of considering raw error, in this work energy-drift will be used as a proxy (as outlined in Section 2.1.2). In this case, stability refers to how the error of the result changes with respect the length of the time domain of the root problem. Convergence measures how many iterations the implementation takes to converge against the coarse and fine discretizations.

Algorithm analysis focuses on two main aspects: how the runtime of the program is affected by changes in size of the input, and how the needed amount of memory is affected by changes in the size of the input. For this work, the "input" will be the coarse and fine discretizations.

Benchmarks

4.1. Numerical Analysis

In order to understand the error of a numerical approximation, it must be compared to a known value. For this analysis, that reference is the simple pendulum. The energy of the simple pendulum, neglecting friction and other dissipative forces, is constant. While the energy of the pendulum itself is directly proportional to its mass, the length by which it hangs, and the acceleration due to gravity, the following error and stability analyses consider the error relative to the the true so that these parameters do not affect the result. Additionally, the considered pendulum begins at its low point with unit velocity.

Because the PA acts as a "meta-algorithm", the underlying integration methods must also be chosen. For these results, the integration schemes for the coarse and fine propagators are the symplectic-Euler and velocity-Verlet methods, respectively. The symplectic-Euler method is chosen for the coarse propagation because it computationally "cheap" while still being symplectic. The velocity-Verlet method is chosen for the fine propagation due to its higher accuracy, while still being computationally inexpensive. While these methods are simliar in their computational cost and accuracy for a single propagation, the multiple resolutions of the time domain provide the ability for the fine propagator to have a higher discretization and thus a much smaller time-step compared to the coarse propagator.

An important item to note is that because this implementation uses the GPU, the precision of the data is restricted to 32-bit floating-point representations i.e. 32-bit floats. On consumer-grade GPUs, significant performance increases are seen between using 32 and 64 (or larger) represenations. Though, when the combination of the coarse discretization and integration method yield low-accuracy data, their error accumulates rapidly. This usally ends with the data at later times becoming too large to be represented by only 32 bits ($\approx 10^{38}$). In any instance, the result could either overflow to their negative maximum, return a "32-bit infinity", or a Not-a-Number (NaN), which is only determined by implementation and hardware specifics.

Another note pertaining to the restrictions imposed by 32-bit floats is that of the so-called *machine epsilon*. For a 32-bit float, the smallest difference between numbers that can be detected is $\approx 10^{-}7$.

4.1.1. Error & Energy Drift

The error in a simulation depends on how close the taken approximation is to the analytic description. For this work, the approximation is completely determined by the time-step; though, the time-step is not determined directly but rather by the discretization of the given time domain. As the PA uses multiple discretizations, there are multiple time steps. The time step used by the coarse propagation only depends on the coarse discretization while the time step used in the fine propagation, because it is simply a fraction of the coarse time step, depends on both the coarse and fine discretizations.

Figure 7 shows how the energy drift of the simulation is affected by the coarse discretization for a particular choice of the fine discretization. There are two key features that should be noted here: the error changes the most between a coarse discretization of 2^2 and 2^3, the difference in the difference of error is non-monotonic for different fine discretizations. In other words, the difference in the change in error for difference coarse discretizations first increases, then decreases for increasing fine discretization. For a fine discretization fo 2^2, the difference in the error for a coarse discretization of 2^2 and 2^3 is big, and it increases with fine discretization until it begins decreasing resulting in the 2^11 (purple) and 2^15 (gold) lines. While it makes sense that this difference should get smaller with a higher fine discretization, the non-monotonicity is interesting and might stem from the $\frac{1}{xy}$ structure of the time step.

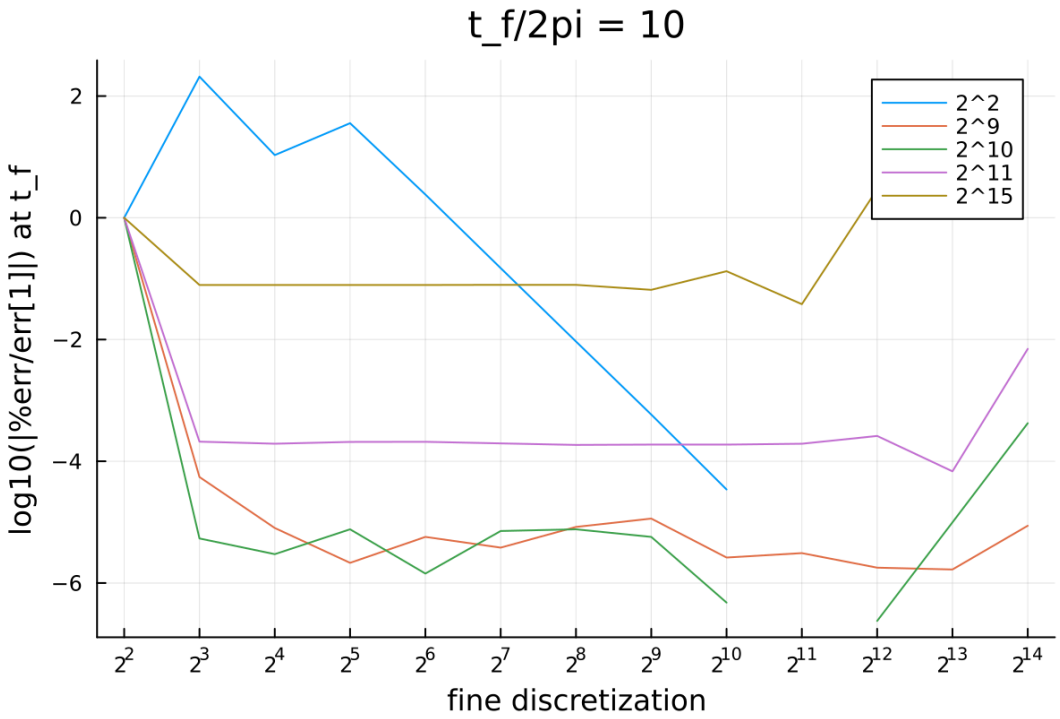

Figure 7: The order of magnitude of the normalized percent error of the final state of the pendulum for different coarse and fine discretizations.

Starts high, and immediately converges

- fine discretization

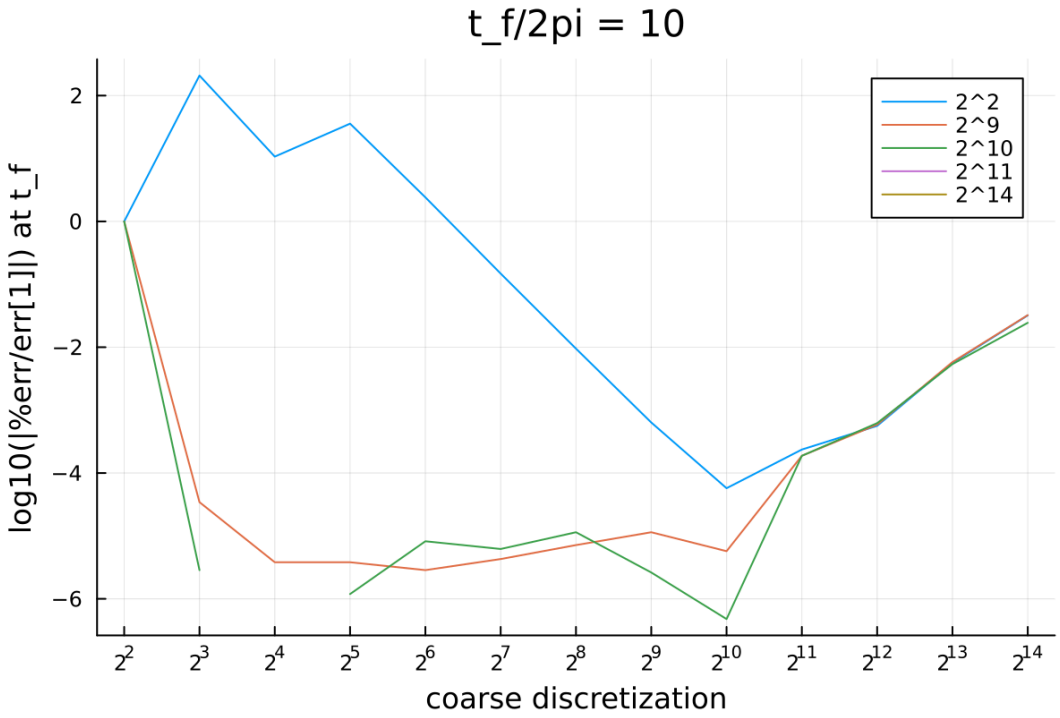

Figure 8:

4.1.2. Stability

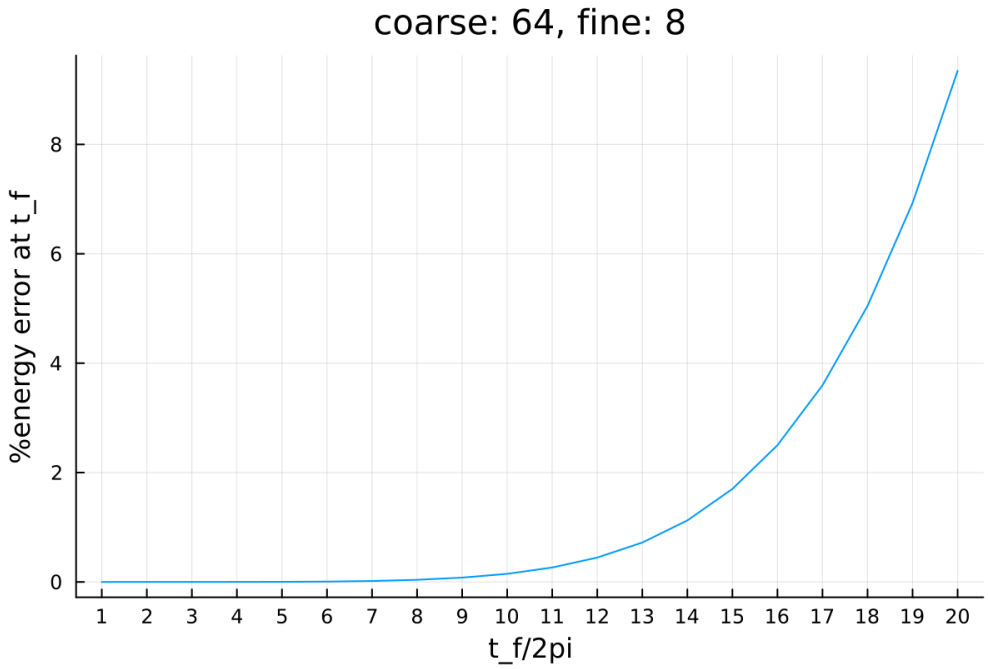

Figure 9:

4.1.3. Convergence

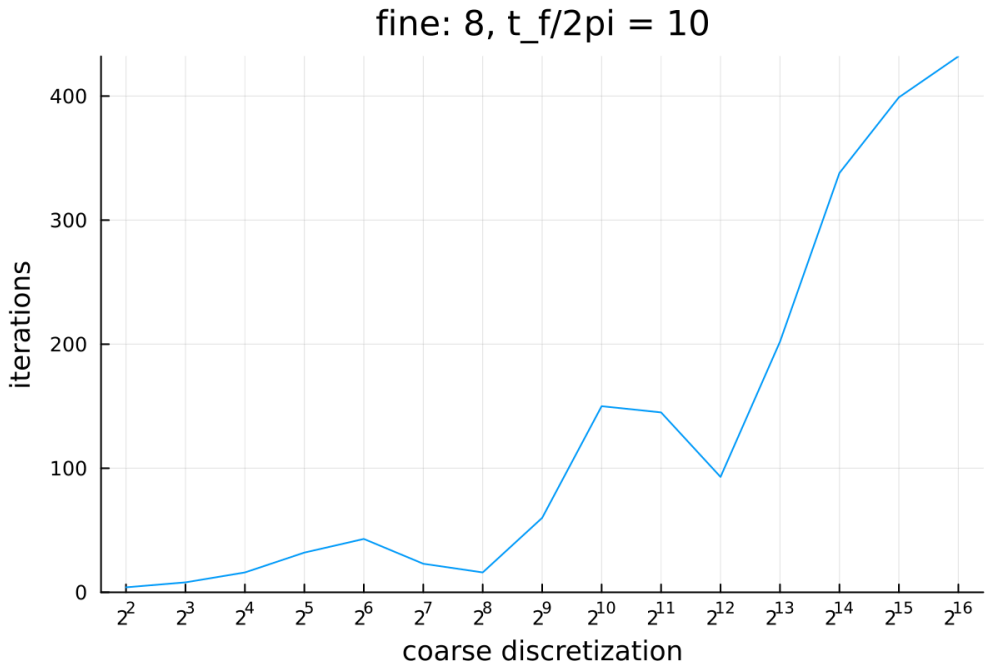

Figure 10:

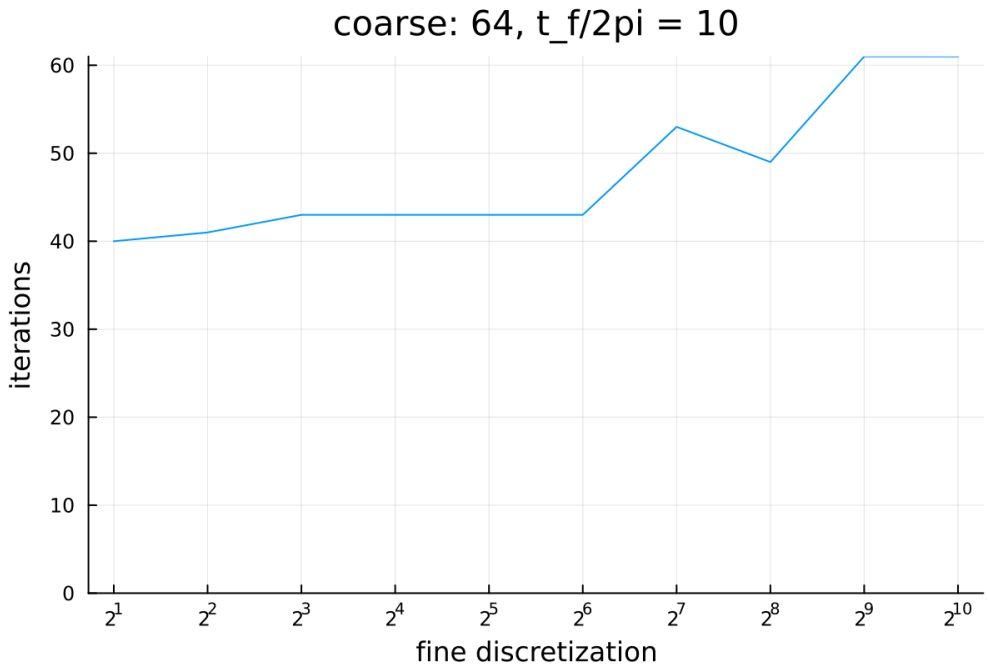

Figure 11:

4.2. Performance Analysis

As noted in Section 2.2, any data that is computed in one memory space must be transferred to another memory space in order for that data to be used in that space. Again, this applies for both GPUs and remote hosts (or more generally processes). Because these transfers happen over either PCI-E or network channels, respectively, they serve as the single greatest bottlenecks for performance in this implementation. The following discussion will address to what extent this implementation is limited by these bottlenecks and some strategies on how to mitigate them.

To highlight the bottlenecks in the PA, we focus our attention back to Algorithm 14. After the director has created the subproblems, it transfers them to each of the worker processes via a network communication. Then the director has to further communicate that the worker execute the PA on the GPU. The worker host then transfers the problem data to the worker device. After the GPU executes the parareal kernel, the new data is transferred back to the host. Finally, the director requests the new data from the host, to which the worker transfers the new data over the network.

These transfer bottlenecks fall into two classifications: host-device, and host-host communication. There are several methods that can be used to mitigate the consequences of the host-device transfer, including taking advantage of dynamic parallelism (where the coarse propagation would be done on the device), pinned/page-locked memory, and zero-copy memory. As for host-host communication, which is done over the network and is the slowest part of the entire implementation, not much can be done to improve it directly other than using faster hardware and/or potentially an optimized cluster configuration, however, the relative efficiency could be improved by transferring as much data at once as possible.

4.2.1. Host-Device Data Transfer

A fundamental bottleneck, as it's part of the algorithm, is the need for all parallel computation to stop and the serial execution of the coarse propagation to complete. While the serial execution is itself a bottleneck in terms of performance, the coarse propagation requires the fine-propagation data on the device, and thus it must first be transferred to the host; similarly, the new data must be transferred to the device after it's been created. These transfers happen over PCI-E, which is really slow compared to the speeds on-device memory transfers. In fact, the bandwidth of global memory on a device can be at least an order of magnitude greater than the PCI-E bandwidth [22].

These slow transfers could be completely circumvented by moving the coarse propagation to a single thread on the device, then using dynamic parallelism to launch the parareal kernel also on the device [23]. While a single device-thread is likely to take more time than a single host-thread when performing the same task, the increase in time from this trade is likely to be less than the decrease in time from not needing to transfer data. Thus the net time difference from this change would be beneficial. Though, this makes sense only when computing everything locally as any distribution to other nodes would requires the use of the host system.

Another point that should be addressed is that the transfer rate between the host and device is not independent of the size of the transfer itself. On some devices, the transfer rate is nowhere near optimal

when then the size of the transfer is below ~2 MB (even with pinned memory), and peak efficiency is only gained with transfers of 16 MB or more [22]. If the device has, say, $N_c = 5 * 10^3$ cores, and they are all being utilized, and in most cases 32-bit floats ($M = 4$ bytes) are used, there is only approximately $N_c M = 4$ MB data to transfer between the host and the device. While this implementation does not take full advantage of the transfer, it would at least be approaching peak efficiency and more performance will be gained with devices with more compute power.

If the number of transfers is unchanged, the implementation could make use of pinned/page-locked memory. Pinned memory being memory on the host which the device can access directly without first requesting the host CPU to retrieve it and send it. Additionally, pinned memory is guarunteed to never be swapped out to disk and thus the device does not need to wait for it to first be transferred to pinned memory [22]. Using this technique could greatly improve performance; the implementation provided in this work does not use it but could be included via library calls [24].

- Zero-copy memory [22]

4.2.2. Host-Host Data Transfer & Cluster Topology

Network is slow. Might drastically depend on cluster topology [25].

4.3. Benchmarks

Benchmarks and other performance evaluations are meaningless without the appropriate context into how the data was gathered. Just as an engineer should include the relevant model of their equipment in their reported data, if the computational scientist wishes their work to be reproducible, such information must be included with the data. As such, the hardware and software specifications that were used in these experiments is presented in Table 1. Further specifications on the GPUs that were used is collected in Table 2. Additionally, inter-node network traffic was routed through a TP-Link TL-SG108 1Gb/s network switch.

	Director	**Worker**
Operating System	Arch Linux (64-bit)	Arch Linux (64-bit)
Kernel	6.14.10	6.14.9
Motherboard	ASRock B760M	ASUS H170
CPU	Intel i7-13700K 24 logical cores @ 5.40 GHz	Intel i5-6500 4 logical cores @ 3.60 GHz
GPU	NVIDIA RTX 3060 Ti	NVIDIA GTX 1660 Super NVIDIA GTX 960
Memory	32 GB @ 6500 MHz	16 GB @ 1600 MHz

Table 1: Specifications for each host/node in the used cluster.

	3060	**1660**	**960**
Cores	4864	1408	1024
Streaming Multiprocessors	38	22	8
Base Clock (MHz)	1410	1530	1176
Boost Clock (MHz)	1665	1785	1201
Memory Clock (Gb/s)	14	14	7
Memory Size (GB)	8	6	4
Memory Bandwidth (GB/s)	448	336	112
FP16, 32, 64 Performance (TFLOPS)	16.2, 16.2, 0.253	10.0, 5.03, 0.157	N/A, 2.46, 0.077
Compatible CUDA up to	8.6	7.5	5.2

Table 2: Hardware specifications for the GPUs used in this cluster.

On the software side, the Julia language was used to encode the calculations. The Julia standard library's Distributed.jl package was used to perform any and all distributed functionality. Additionally, the CUDA.jl package was used to facilitate the implementation of GPU-based calculations. The versions of these packages are detailed in Table 3.

Julia Runtime	1.11.5	LLVM	libLLVM-16.0.6
Distributed.jl	1.11.0	CUDA.jl	5.7.3

Table 3: The software versions used in the calculations presented in this work.

5. Conclusion

- Equations of motion can now benefit from parallel solvers.
- Certain problems are well-suited to a divide-and-conquer approach.
- Problems with "doubly parallel" characteristics can leverage both local and distributed parallelism, achieving significant computational efficiency.
- These advancements pave the way for modeling acoustics in expanding volumes.

5.1. Future work & Possible Optimizations

- Krylov enhanced subspaces
- CUDA dynamic parallelism
- Implement with C, Fortran, CUDA, NVSHMEM, MPI
- Make gpu-backend-agnostic with KernelAbstractions.jl
- this physics could be better done with PFASST
- non-dimensionalize everything following [26]
- use a variable size time discretization algorithm, then base integration of those differences
- Use dynamic parallelism to avoid the cpu having to launch the kernels
- Use dynamic parallelism to even perform the coarse propagation
- Look into effects of cluster topology

Bibliography

[1] N. Chapman, Ground state phases of a quasi-2D BEC of rigid-rotor molecules via Bogoliubov mean-field theory, (2019).

[2] L. D. Landau and E. M. Lifshitz, *Mechanics, Third Edition: Volume 1 (Course of Theoretical Physics)*, 3rd ed. (Butterworth-Heinemann, 1976).

[3] S. A. Orszag, Numerical Methods for the Simulation of Turbulence, The Physics of Fluids **12**, (1969).

[4] C. R. (https://scicomp.stackexchange.com/users/18981/chris-rackauckas), What does "symplectic" mean in reference to numerical integrators, and does SciPy's odeint use them?, (n.d.).

[5] C. Cramer, *Essentials of Computational Chemistry: Theories and Models* (Wiley, 2013).

[6] H. Bock and K. Plitt, A Multiple Shooting Algorithm for Direct Solution of Optimal Control Problems*, IFAC Proceedings Volumes **17**, 1603 (1984).

[7] M. Harris, How to Optimize Data Transfers in CUDA C/C++, (2012).

[8] M. Harris, An Even Easier Introduction to CUDA, (2017).

[9] M. Harris, CUDA Pro Tip: Write Flexible Kernels with Grid-Stride Loops, (2013).

[10] D. Singal, N Ways to SAXPY: Demonstrating the Breadth of GPU Programming Options, (2021).

[11] J. Barnes and P. Hut, A hierarchical O(N log N) force-calculation algorithm, Nature **324**, 446 (1986).

[12] T. Hamada, K. Nitadori, K. Benkrid, Y. Ohno, G. Morimoto, T. Masada, Y. Shibata, K. Oguri, and M. Taiji, A novel multiple-walk parallel algorithm for the Barnes–Hut treecode on GPUs – towardscost effective, high performance N-body simulation, Computer Science - Research and Development **24**, 21 (2009).

[13] J.-L. Lions, Y. Maday, and G. Turinici, Résolution d'EDP par un schéma en temps «pararéel », Comptes Rendus De L'académie Des Sciences - Series I - Mathematics **332**, 661 (2001).

[14] M. J. Gander and S. Vandewalle, Analysis of the Parareal Time-Parallel Time-Integration Method, SIAM Journal on Scientific Computing **29**, 556 (2007).

[15] C. Pinter, *A Book of Set Theory* (Dover Publications, 2014).

[16] B. W. Ong and R. J. Spiteri, Deferred Correction Methods for Ordinary Differential Equations, Journal of Scientific Computing **83**, 60 (2020).

[17] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms, Fourth Edition* (MIT Press, 2022).

[18] N. Giordano and H. Nakanishi, *Computational Physics* (Pearson/Prentice Hall, 2006).

[19] N. Chapman, PararealGPU.jl, (n.d.).

[20] W. Stallings, *Operating Systems: Internals and Design Principles* (Pearson Education, 2011).

[21] C. Li, C. Ding, and K. Shen, *Quantifying the Cost of Context Switch*, in *Proceedings of the 2007 Workshop on Experimental Computer Science* (Association for Computing Machinery, San Diego, California, 2007), p. 2–es.

[22] S. Cook, *CUDA Programming: A Developer's Guide to Parallel Computing with Gpus*, 1st ed. (Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2012).

[23] A. Adinets, CUDA Dynamic Parallelism API and Principles, (2014).

[24] T. Besard, C. Foket, and B. De Sutter, Effective Extensible Programming: Unleashing Julia on GPUs, IEEE Transactions on Parallel and Distributed Systems (2018).

[25] Y. Deng, M. Guo, A. F. Ramos, X. Huang, Z. Xu, and W. Liu, Optimal low-latency network topologies for cluster performance enhancement, The Journal of Supercomputing **76**, 9558 (2020).

[26] H. Langtangen and G. Pedersen, *Scaling of Differential Equations* (Springer International Publishing, 2016).